MRIDANGA SEVA · FOR CODE MAINTAINERS (SEVA)

The Mridanga Seva app, explained

What the class app does and who uses it; how it is built and why; how a sign-up, a check-in, a follow-up call, a practice session or a parent's email really travels through the code; how children's data is protected; and how a change goes safely from a laptop to the class's phones.

Written for beginners and college students: no app or database background needed, every word explained the first time. Facts taken from the code on branch `main` at commit `b2e5fb5`, 24-10-2026, four days after the class started using the app (20-10-2026).

Contents

1. What the app does, and who uses it
2. The building blocks and why we chose them
3. Five ideas you need first
4. The big picture
5. Journey: signing up and being accepted
6. Journey: a child signs up, the parent consents
7. Journey: QR check-in, check-out, attendance
8. Journey: follow-up calls and student status
9. Journey: learning: progress, lessons, assessments, level-ups
10. Journey: practice at home
11. Journey: news, notifications, events and polls
12. Running the class: fund, instruments, roster, reports
13. Parents by email, and Ishtagoshti for everyone
14. Security: roles, row-level security, keys
15. Privacy and the law (DPDP)
16. The data model: tables, migrations, functions, jobs
17. Code map: which folder does what
18. Screens and their files
19. Testing and CI
20. Release and operations
21. How decisions are recorded
22. How to contribute safely
23. What is not built yet, and Phase 3
24. Glossary

> **One sentence summary.** One TypeScript code base (Expo / React Native) gives an Android app and a web version; both talk to one Supabase project, where a PostgreSQL database holds every record **and every rule** about who may see or change it, so no screen, old app or typed web address can get around the rules.

> **How to read this.** Screen names are written as the app shows them in English, with where to find them, e.g. *Welcome desk (Students tab)*. The docs and code comments also use short screen codes (C2, G10…) from the screen list the team approved on 28-09-2026; section 18 maps the names to the code files. A NEXT tag marks something that is on `main` and on TEST but not yet in the class's (LIVE) app on 24-10-2026; OFF marks a feature that is built but switched off until the Guru switches it on.

1. What the app does, and who uses it

Mridanga Seva is the app of an ISKCON mridanga (khol) class in Hyderabad. The class is **drop-in**: students come any time between the afternoon and evening, not in fixed batches. So the app thinks in **visits** (a check-in and a check-out), not in roll calls. It replaces an Excel sheet and about ten WhatsApp groups, and it keeps a proper record of consent for students under 18.

Who uses it

Who	Role in the database	What they do in the app
Students (many under 18)	`student`	Show *My QR card* at the door; see their attendance, progress and lessons; practise at home with the metronome and taal player; read News; answer events and polls; send assessment recordings; study slokas.
Coordinators (teachers who run the daily class)	`coordinator`	Register students at the *Welcome desk*; check students in and out; tick syllabus progress; make follow-up calls; post News; review assessments; lend instruments.
The Guru (shown as *Facilitator* on the English screens)	`guru`	Everything a coordinator does, plus: gives roles, decides level-ups, edits the syllabus, settings, centres; reads the change history and access log; answers data requests.
Someone who just signed up	`pending`	Sees *Waiting for access* until the desk accepts them; meanwhile may use *While you wait* (practice tools, the mantra, Beginner lesson videos).
Ishtagoshti members (public, free)	`pending` + a row in `ig_subscribers`	Read the class's slokas (verses) and keep their own notes. Nothing else of the class.
Parents	no login	Get emails: the consent code for a child's home sign-up, and (when switched on) a check-out email and a weekly summary. They consent on a web page, *Consent for your child*.
The door tablet	`kiosk` (exists in the database only)	**Not built yet.** The role and its rights exist; the app shows "The door tablet screens come in a later version". Today a coordinator's phone at the door does the scanning (the "door phone").

Where it runs

> **Android app** (an APK installed from a link, not from the Play Store). Used by most staff and Android students. Gets push notifications. Updates itself over the internet.

> **Web version** (`mridanga-seva.pages.dev`). Same code, in a browser. iPhone users add it to their home screen, because an App Store listing costs US$99 a year. Works partly offline. No push notifications yet.

There is also a separate **public website**, `mridangaseva.com` (folder `website/`): plain HTML in English, Telugu and Hindi, with the classes, how to join, the app links and the **privacy notice**. It shares no code with the app.

The main modules

Area	What it covers
Registration	Sign-up, email confirmation, About you, the Welcome desk, consent for minors, printed QR cards
Attendance	QR scan or tap, In class now, Check out all, location flag, history, reports
Follow-up	Automatic Irregular / Inactive status, call tasks for the mentor, Record a call
Learning	Three levels (Beginner, Intermediate, Advanced), syllabus ticks, lessons (YouTube, PDFs), assessments, level-ups
Practice	Metronome, taal player, two-head view of the drum, practice timer and log, games (Tap pad, Which bol?, Echo me)
Communication	News (announcements), groups, private replies, notifications inbox, push, events, polls
Running the class	Class fund, instruments with QR labels, seva roster, material suggestions, settings, centres
Ishtagoshti	Sloka study: themes, sloka of the day, recitations, notes, memorised ticks; free public sign-up
Privacy	Consent register, access log, change history, My data and rights, data requests, erasure, retention

2. The building blocks and why we chose them

The full reasoning is in `docs/WHY_THIS_STACK.md` and `docs/DECISIONS.md` #1, #5, #6. In short: a drop-in class of about 200 people, many of them children, run by volunteers with **no budget** ("Phase 1 must cost nothing"), most members on iPhones, and few maintainers.

Block	What it is	What it does for us	Why this one
TypeScript	JavaScript with types	The language of the whole app	Widely known; the type check catches many mistakes before a phone does
React Native + Expo (SDK 57)	Write a phone app with React; Expo adds tools and ready parts	One code base → Android app *and* web version	No Mac or Android Studio needed; camera, audio, location, notifications ready-made
Expo Router	Every file in `app/src/app/` is a screen	Navigation, web addresses, protected areas per role	The folder *is* the map of the app
Supabase	A hosted PostgreSQL database plus login, file storage, server functions	All records, all rules, logins, files, scheduled jobs	Free plan; standard Postgres we can take elsewhere
PostgreSQL	A relational database (tables of rows)	Row-level security, triggers, functions, checks at commit	MySQL has no row-level security; a document database makes attendance reports hard
Row-level security (RLS)	Per-table rules: who may read or change which rows	A student sees only their own record, whatever the app asks	The rule lives in one place and holds for every screen
Edge Functions	Small TypeScript programs on Supabase's servers	`notify-announcements` (push), `notify-parents` (emails)	The only code that holds the secret key, inside Supabase
pg_cron	A timetable inside the database	17 jobs: statuses every morning, push every minute, clean-ups at night	No separate server to keep alive
EAS Build / EAS Update	Expo's cloud services	Builds the Android APK; sends new JavaScript to installed apps	Free: 15 builds a month; updates for 1,000 people a month
Cloudflare Pages	Static web hosting	Serves the web version and the public website	Free, fast, our own security headers
Brevo	An email-sending service	Sign-up and password emails; parent emails	Free plan: 300 emails a day
Expo push + Firebase	Push notification delivery	Notifications to Android phones	Free
YouTube	Video hosting	Lesson videos; the app stores only the links	Free; keeps big files out of our 1 GB of storage
GitHub	Code hosting, public, MIT licence	The code, pull requests, the CI checks	Anyone can read exactly what the app does

What these choices cost us, and how we handle it

Cost	How we handle it
The free database pauses after 7 days without use	Daily class use keeps it awake; in holidays it is opened every Monday with the backup
The free plan has no downloadable backups	A weekly encrypted backup by hand, with a restore drill (section 20)
500 MB of data, 1 GB of files	Videos stay on YouTube; recordings are deleted 30 days after review
iPhone users get a web app, not an App Store app	The same code can build an iOS app once the US$99 a year is agreed
The rules are written in SQL, which fewer volunteers know	One numbered file per change, its reason in DECISIONS.md, 2,224 automatic database checks

Versions on main (app/package.json): expo ~57.0.27, react-native 0.86.3, react 19.2.3, expo-router ~57.0.25, @supabase/supabase-js ^2.118.0, i18next ^26.4.7, typescript ~6.0.3; eas-cli pinned at exactly 24.12.1.

3. Five ideas you need first

Idea 1: the app keeps almost nothing of its own

The phone or browser draws screens and sends requests over the internet. The **records** live on the server, in the database. So every coordinator sees the same students on any phone. (A few things are kept on the device on purpose: the signed-in person's own profile so the app opens at once, My QR card so it works without signal, the practice taals, and check-ins or practice saved while offline until they can be sent.)

Idea 2: tables and rows

A database is a set of very strict registers. Each kind of record is a **table** (like a sheet in Excel): `students`, `visits`, `call_logs` … Each record is a **row**. A row in `visits` is one check-in, with its check-out time filled in later. Tables point at each other by **id**: a visit holds the `student_id` of its student.

Idea 3: the rules live in the database (the most important idea)

A screen can hide a button, but a screen is not a lock: someone could use an old app version or type a web address. So **every rule that protects people or records is enforced by the database itself** (DECISIONS #5). Three tools do it:

Tool	What it is	Example from our app
Row-level security (policies)	Written rules per table: who may read, insert, update, delete which rows	A student asking for "all students" gets only their own row
Triggers and constraints ("guards")	Checks that run whenever a row is saved; they refuse a bad save	A roll number can never change; Paused and Left only through a logged call
Database functions	Actions that change several things at once, all or nothing, checking who calls	`scan_qr` checks a student in, makes them Active and closes their call tasks

Idea 4: migrations

The database is not built by clicking. It is built by **migrations**: numbered SQL files in `supabase/migrations/`, run in number order (0001, 0002 … 0061). A migration that has run is **never edited**; a change is a new file. Running the same files on an empty database rebuilds the whole schema, which is how the tests work.

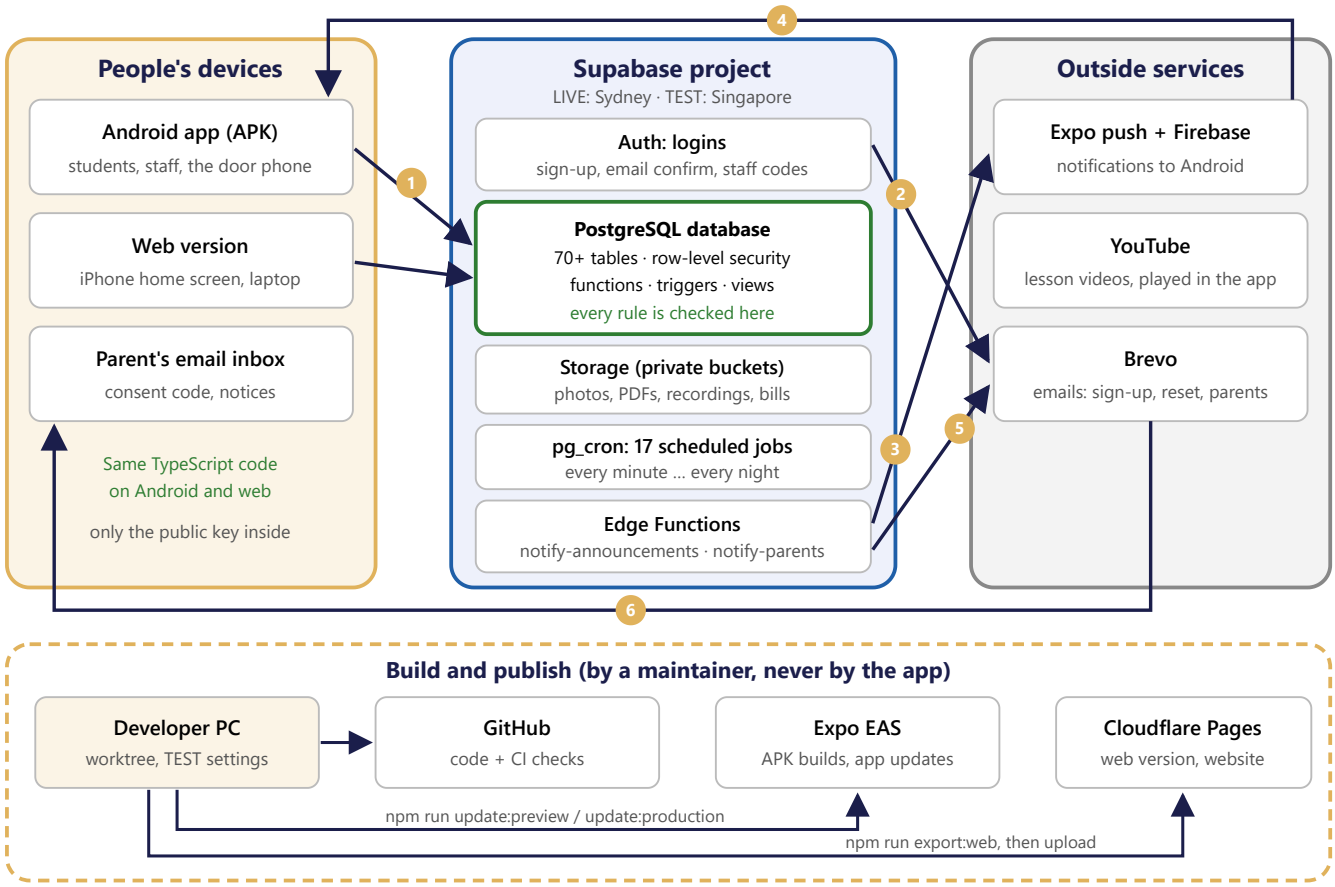
Idea 5: TEST and LIVE

There are two separate Supabase projects with the same schema:

	TEST (vfhuqvk…, Singapore)	LIVE (qeozvv…, Sydney)
Holds	Made-up students (`supabase/seed.sql` : 15 fictional students, `example.com` emails)	The class's real records, children included
Web site	`mridanga-seva-test.pages.dev`	`mridanga-seva.pages.dev` (later `app.mridangaseva.com`)
Android	the *preview* APK and update channel	the *production* APK and update channel
Who touches it	Builders, testers, volunteers	The class. Changes only by Praveen, after TEST, after a backup

> **Never LIVE.** Real records are never used for testing and made-up students never go into LIVE (on 29-09-2026 test students were found in LIVE by mistake and removed the same day). Scripts check that a test build talks only to TEST and a live build only to LIVE (section 20).

4. The big picture

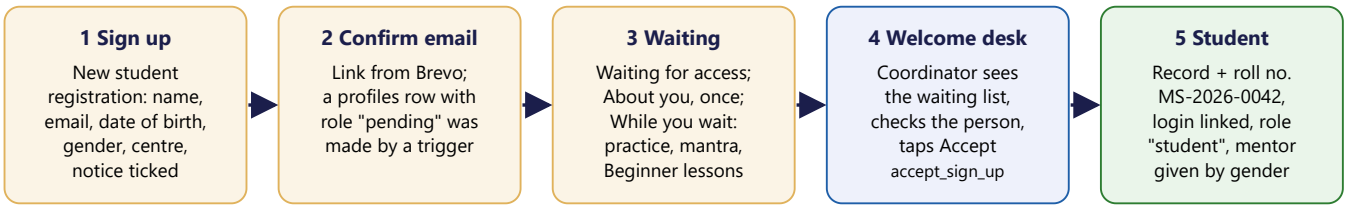


#	What travels
1	The app's reads and writes, over HTTPS, with the **public** key and the person's login token. The database checks each one against row-level security and the function's own role check.
2	Supabase Auth sends sign-up confirmation and password-reset emails through Brevo (SMTP).
3	Every minute a pg_cron job calls the Edge Function `notify-announcements` when something waits to be pushed; it hands the messages to Expo's push service.
4	Expo passes them to Google Firebase, which delivers them to the Android phones. The web version gets none (yet).
5	Parent emails (consent code, check-out notice, weekly summary) go from `notify-parents` to Brevo's API.
6	Brevo delivers the emails to the parent's (or the new user's) inbox.

Read it like this: phones and browsers talk to Supabase directly. Supabase checks who you are (Auth), keeps the records and the rules (database) and the files (Storage), and runs jobs on a timetable (pg_cron). Those jobs call two small programs (Edge Functions) that hand notifications to Expo and emails to Brevo. Videos stay on YouTube. In the bottom lane, a maintainer's PC sends code to GitHub, app builds and updates through Expo EAS, and the web version to Cloudflare.

5. Journey: signing up and being accepted

Most students create their own login at home. The desk then **accepts** them with one tap, using what they typed, so the form is filled once, by the student (DECISIONS #263).



Screens: sign-up.tsx → pending.tsx / about-you.tsx / waiting/ → staff/register.tsx → student/(tabs)

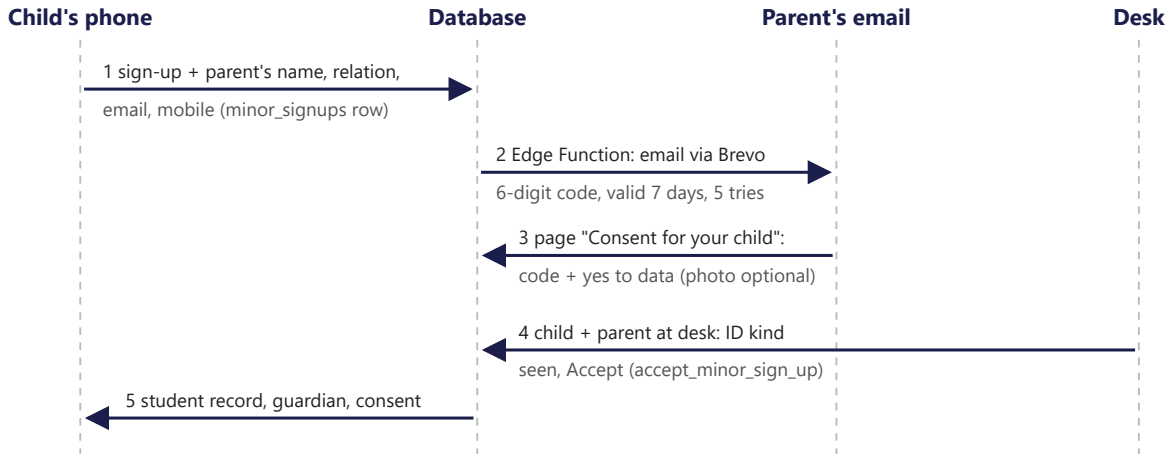
1. **Sign up** (`app/src/app/sign-up.tsx`). Name as on a government ID, email, password, date of birth, gender, mobile, centre (country → city → centre) and a required tick "I have read the privacy notice". The lists come from `sign_up_choices()`, the **only** function someone not signed in may run. Under 18 → the extra parent part (section 6).
2. **Confirm the email.** Supabase sends a link; the trigger `handle_new_user` already made a `profiles` row with role `pending`. Logins are linked only after the email is confirmed; otherwise anyone could claim a record by typing its email (DECISIONS #13).
3. **Waiting for access** (`pending.tsx`). *About you* opens once (emergency contact, how they found the class…, table `person_details`). An adult (or a child whose parent consented) may already use *While you wait* (`app/src/app/waiting/`, function `practice_while_waiting()`, 0057).
4. **Welcome desk (Students tab)** (`staff/register.tsx`). The coordinator sees the confirmed sign-ups of their centre (`waiting_sign_ups()`) and possible duplicates (`possible_duplicates`), and taps **Accept**. For someone without a phone or login there is still the full form (`register_student`).
5. **Student.** `accept_sign_up(profile)` writes the `students` row; a trigger gives the roll number `MS-<year>-<4 digits>`, frozen for ever and never reused; About you moves to the record; a mentor of the same gender is assigned; the login becomes `student`, and the person's app opens the student screens.

> **Why a separate "pending" role?** Until the desk has seen the person, a login can read and edit only its own profile. The test net in `supabase/tests` checks, table by table, that a pending login reads nothing else.

Other paths to the same link: a coordinator saving an email on an existing record links a waiting login with that email (trigger `students_link_login`); the Guru links by hand (`link_student_login`). A minor's record is never linked automatically (0047): only by the parent-consent path or the Guru.

6. Journey: a child signs up, the parent consents

Most students are under 18. India's DPDP Rules (rule 10) ask for **verifiable consent of a parent** before a child's data is processed. Since migration 0051 a child can start at home, and the parent consents by email (DECISIONS #399). Before that, "under 18" meant "come to the desk with a parent", and on 20-10-2026 several children tried at home and gave up.

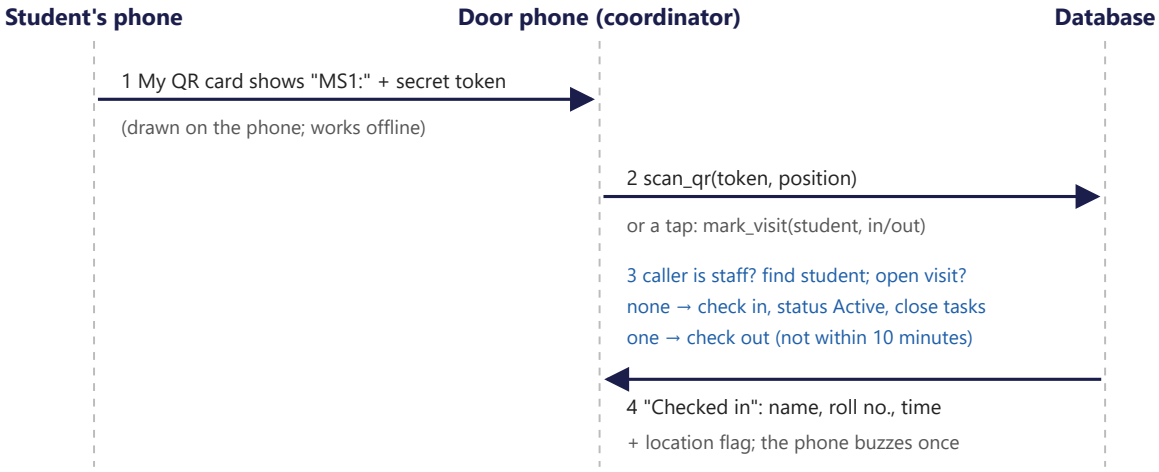


1. The sign-up form sees a date of birth under 18 and asks the **parent's** name, relation (mother, father, guardian), email and mobile. The database keeps them in `minor_signups`, a table no app user can read.
2. The job `mridanga-parent-notices` (every minute) calls the Edge Function `notify-parents`, which claims a **new 6-digit code** (stored only as a hash) and sends the email through Brevo, in the child's app language. Limits: 5 emails per sign-up, 3 a day per address, 60 a day for the whole app.
3. The parent opens *Consent for your child* (`app/src/app/parent-consent.tsx`, open without a login), types the code, declares they are the parent or legal guardian and 18 or over, and says yes to the data (a photo is a separate, optional yes). `give_parent_consent` records it with the privacy-notice version shown.
4. Only now the child appears on the *Welcome desk*. They come with the parent; the coordinator looks at the **parent's ID document** and records only its **kind** (Aadhaar, PAN, ...), never its number, then taps Accept.
5. `accept_minor_sign_up` writes the student (linked to the child's login), the guardian and the consent of method `email_code`, and deletes the request. Without the parent's yes within 30 days, or without the desk within 180 days, a nightly job erases the whole sign-up (`mridanga-minor-signups`).

The rule that holds everywhere

Whatever the path (this one, the desk form, the Excel import, the dashboard), a trigger checked **at commit** refuses a student under 18 on record without a current data consent given through one of **their own** guardians, and a guardian with a phone (`minor_rule_gap`, errors `minor_needs_consent`, `minor_needs_guardian`; DECISIONS #16, #272). The desk form's equivalent for a parent who signs on paper: the coordinator ticks that the signed form was received (`signed_form`).

7. Journey: QR check-in, check-out, attendance



1. **My QR card (My QR tab)** shows a QR code holding `MS1:` and the student's secret `qr_token`, not the roll number, so nobody can make another student's code by guessing (DECISIONS #17). The last one loaded is kept on the phone. A student without a phone gets a printed card with the same code (*Print QR cards*, 0045); a lost card is re-issued with `reissue_qr_code` and the old one stops working.
2. The coordinator opens **Check-in (tab)** (`staff/mark-attendance.tsx`) and scans with the camera (`expo-camera`; on the web, the browser's reader or `zxing-wasm` on iPhone Safari). Without a camera they search the name and tap *Check in* / *Check out*.
3. `scan_qr` → `toggle_visit`: check in if the student has no open visit, else check out, all in one transaction. A second scan within 10 minutes of a check-in answers "already in", so a second phone or a retry does not end the visit. `mark_visit` does exactly what the button says and locks the student's row while it works.
4. The answer carries the student's name, roll number and time. Any check-in makes the student **Active** and closes their open follow-up tasks.

The location flag (0024, DECISIONS #70)

A check-in also sends the **marking phone's** position (foreground only, at most 8 s). The database compares it with the centre's point and radius (set on *Centres*) and stores **only the result and the distance**, never the position: `inside`, `outside`, `refused`, `no_fix`, `no_location`, `no_area`. Outside or unknown is **saved anyway but flagged** for the Guru and counted in the reports.

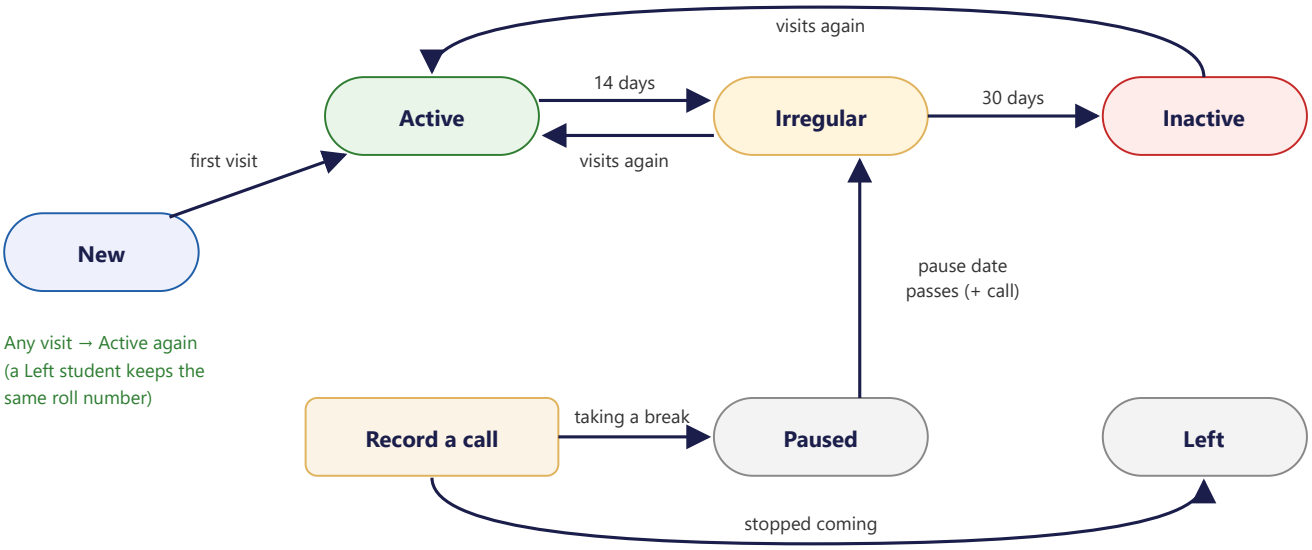
Closing time, and days without signal

- **In class now** lists open visits; at closing time **Check out all** (`check_out_all`) closes them.
- Every hour the job `mridanga-close-visits` closes any visit still open an hour after its centre closed, at the closing time (21:00 India time for Abids). Each centre has its own time zone (0033).
- Staff may correct only a visit's **times**, and each correction is in the change history (0030).
- NEXT **No signal at the door** (DECISIONS #593): a scan or tap is kept on the door phone (`lib/offline-queue.ts`) and sent when the signal is back, the same class day only, with a request id so it is saved once. A scanned code is re-sent only as a check-in, because `scan_qr` toggles.

The student sees their visits in *My attendance*; staff see them on the student's page and in *Reports* (`class_report`, with a CSV download).

8. Journey: follow-up calls and student status

In a drop-in class nobody notices when a student quietly stops coming. So the database watches the days since the last visit, and gives the student's **mentor** (a coordinator) a call to make.



1. **Every morning at 06:00 India time** the job `mridanga-status-refresh` runs `refresh_student_statuses()`: no visit for 14 days → **Irregular** and a `call` task for the mentor, due in 3 days; 30 days → **Inactive**. The numbers are rows in `settings` (the Guru changes them in *Settings*), not code.
2. The coordinator opens **Follow-ups (tab)** (`staff/(tabs)/follow-up.tsx`): students grouped as *needs the Guru*, *call due*, *call later*, and Irregular or Inactive with no call planned.
3. **Record a call** (`staff/call/`) has buttons that open the phone's dialler for the student or, for a minor, the parent. Afterwards: the outcome, a reason from the list (studies, work timing, moved, health…), a comment and, for "coming back" or "taking a break", a date.
4. `log_call` saves the call and applies the outcome: *taking a break* → Paused until the date; *stopped coming* → Left; *coming back* → a new call for the day after the date; *not reachable* → a retry, and after several failed tries in the same absence the task is **escalated** to the Guru.

> **Why only through a call?** A trigger refuses setting Paused or Left directly (DECISIONS #4), so nobody is dropped from the class without a reason and a comment. Every status change is kept in `status_history`, every call in `call_logs`. Call notes are private: coordinators read them only through `get_call_notes`, which writes a row in the **access log** (section 15).

9. Journey: learning: progress, lessons, assessments, level-ups

Levels and the syllabus

There are three levels: **Beginner, Intermediate, Advanced** (table `levels`). Each has an ordered list of `syllabus_items`, which the Guru edits in *Lessons* (`staff/levels/`); items are retired, never deleted, so no tick is lost (DECISIONS #44).

Mark progress (ticks)

When a student shows an item in class, a coordinator ticks it in *Mark progress* (`staff/syllabus/`): a row in `student_progress` with the date and an optional remark. The trigger `student_progress_guard` writes the **real** ticker's id whatever the app sends, refuses a future date, and freezes the tick except its remark; every tick and untick is copied to the change history (DECISIONS #22). The student sees it in *My progress*.

Lessons (materials)

Each level or item can have lessons: a YouTube link, a video file link, a PDF or a photo (`materials`, files in the private bucket `material-files`). A student sees approved lessons up to their own level. *Lesson video* plays YouTube in YouTube's own embedded player (never altered, as YouTube's terms require) with speed, A-B loop and, for the team's own videos, mirror and camera-angle zoom. Coordinators can *suggest* a lesson; the Guru adds or declines it.

Assessments

Step	Who, where	What happens in the database
1 Create	Guru: instructions, level, rubric, files; a draft until sent	`assessments`
2 Give	Coordinator: notes, due date, picks students	`release_assessment` → one assignment each + a notice
3 Send	Student: records with the phone or in the app, sends a file (≤ 50 MB) or link	`submit_assessment` checks the file is theirs, bucket `assessment-files`
4 Track	Coordinator: Not seen / Seen / Submitted / Reviewed / Redo / Late; Remind	view `assessment_tracker`; daily reminder at 09:00
5 Review	Coordinator: plays it, scores each rubric line, comment or voice note	`review_submission`; recording deleted 30 days after review

Level-ups (promotion)

The app never promotes anyone by itself (DECISIONS #53).

1. `promotion_criteria` checks a student against the Guru's settings: the level's syllabus ticked, 8+ visits in 8 weeks, an accepted level-up recording.
2. A coordinator recommends the student with a reason and asks other coordinators (`nominate_for_promotion`).
3. They answer Ready / Almost / Not yet with a comment; at 2 answers it waits for the Guru (*Level-up queue*).
4. The Guru decides: **Promote** (`students.level_id` + a `level_history` row), **Not yet** with a date, or **More feedback**. A trigger refuses any other change of level.

Screens: `staff/assessments/`, `student/assessments/`, `staff/promotion/`, `staff/levels/`, `staff/materials/`, `screens/lesson-player.tsx`. Tables: see section 16.

10. Journey: practice at home

Practice tools (`screens/practice-tools.tsx`, routes `student/practice.tsx` and `staff/practice.tsx`) turns the phone into a practice partner. Nothing in these tools is sent anywhere except the practice log.

Tool	What it does	Where the code is
Metronome and taal player	Loops a beat or a taal's bols at a tempo; the screen lights the beat	`lib/practice-pattern.ts`, `practice-player.ts`
Two-head view	A drawing of both drum heads lighting the stroke being played: a student watching the teacher cannot see both heads	`components/two-head-view.tsx`, `lib/bols.ts`
Practice timer and log	Start / Stop; Stop saves the minutes (`log_practice`); "How did it go?" hard / okay / good	`lib/practice-timer.ts`, *My practice log*
Practice of the week	Per level: weekly goal (default 4 days × 15 min), up to 3 things to practise; a student's own goal	`practice_plans`, `practice_goals` (0052)
Mantra before practice	A mantra to read first, in Devanagari, Telugu or Roman letters	`practice_mantras`
Record myself	Records with or without the metronome, plays back beside it; takes stay on the phone	`lib/recording.ts`, `lib/my-recordings.ts`
Play and learn	*Tap pad* (play the taal on two pads, scored ★ to ★★★), *Which bol?* (ear quiz), *Echo me* (call and answer), *Learn more* (links the Guru trusts)	`lib/tap-grading.ts`, `ear-quiz.ts`, `echo.ts`; table `practice_links`
Milestones	Streaks and totals worked out on the phone from `practice_history()`	`lib/milestones.ts` (0056)

How the sound keeps time

> **In the browser** (`practice-audio.web.ts`): Web Audio. A timer every 25 ms looks 150 ms ahead and schedules each sound on the **audio clock**, not on the timer. Measured on 3-10-2026: 183 clicks in 136 s, worst error 0.16 ms, none late, even in a background tab.

> **On the phone** (`practice-audio.ts`, `expo-audio`): one cycle is mixed into a WAV file and played on loop, so the audio hardware keeps the time. A tempo change writes a new file and seeks to the same beat.

Who sees the practice

The student sees their own week and milestones; staff see each student's week in *Practice this week* (`staff_practice_week()`, least practice first, so the mentor knows whom to encourage). The "How did it go?" rating is new data about a child, so the privacy notice names it.

NEXT Without signal, Stop counts the minutes there and then and keeps the entry on the phone; it is sent once the signal is back (DECISIONS #593, #594).

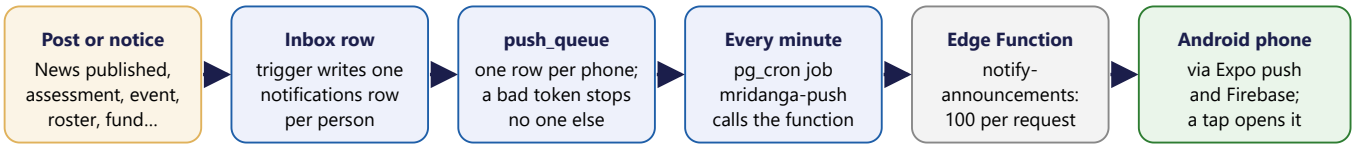
11. Journey: news, notifications, events and polls

News (announcements)

A coordinator or the Guru opens **News (tab)** → New: title, message, up to 3 photos or PDFs, who it is for (all students, one level, my mentees, staff only, or a *group*), pin or not, now or at a later time. The app shrinks photos, uploads them to the private bucket `announcement-files` and saves the row in `announcements`; triggers check it and record the author. A file opens only for people allowed to read its announcement, through a signed link that expires (DECISIONS #32).

- **Seen by N of M**: opening a post writes a read receipt; the views `announcement_seen` and `announcement_audience` count them, with the names of those who have not seen it.
- **Private replies**: a reply is read only by its writer, the author and the Guru (DECISIONS #29).
- **Groups** replace the WhatsApp groups; a group is switched off, never deleted.

How a notification reaches a phone



finish_push records each row as sent, to retry later, or refused; a retry never sends twice (0031)

The **inbox** (the bell on the home header, *Notifications*) is filled for everyone, with or without push, so the web version's users see new things when they open the app. The phone registers its Expo push token with `register_push_token` (at most 5 phones a person). Phones unused for 90 days lose their token NEXT (retention job, 0060).

Events and polls

Events: a programme with date, place and audience; people answer whether they are coming (`rsvp_event`); staff mark performers and who came; "Add to calendar" makes a calendar file. *Polls*: one question, people vote through `vote_poll`; nobody, not even the Guru, can read the votes table directly, and in an **anonymous** poll the functions never show anyone what a person chose (DECISIONS #61). A daily job at 09:00 sends the day-before and closing-day reminders.

12. Running the class: fund, instruments, roster, reports

Class fund (*Class fund*, 0026)

- An income and expense **ledger** in `fund_entries`, amounts in whole **paise** (integers, never decimals), shown as ₹1,25,000.50. Bills go to the private bucket `fund-bills`.
- **Maker-checker**: larger expenses wait for approval, and **nobody approves their own entry**. Treasurers are coordinators the Guru marks.
- **Append-only**: a trigger refuses deletes and edits for every role, the dashboard included. A mistake is a *reversal* row with a negative amount.
- The app writes the ledger **only through functions** (`record_fund_entry`, `decide_fund_entry`…), so the rules hold on every path. Students and parents see no money.

Instruments (*Instruments*, 0023 + 0035)

- Every seva asset has a code (e.g. KHOL-007) and an unguessable `asset_token`. *Print labels* prints an A4 sheet of 18 QR labels from the browser; the label is a web link, so any phone camera opens it, and the app shows it to staff only.
- Lending and taking back are functions (one open loan per item; no damaged item out); every condition seen is a row in `inventory_checks`. *Count instruments* runs a stocktake.

Seva roster and suggestions

Seva roster: the Guru plans duty shifts with their people in one call (`save_duty_shift`); a job at 18:00 reminds everyone on tomorrow's duty. *Suggestions*: a coordinator suggests a lesson; it stays hidden from students until the Guru adds it.

Reports, settings and set-up (Guru)

Screen	What it is for
Reports / *My reports*	Visits per week and month, new joiners, calls made and due, students by status, location flags; CSV download. A coordinator sees their own mentees
Settings	Follow-up day limits, what "this week" means, the open window, promotion criteria, the parent-email switches
Centres	Each class location: address, map point, radius, opening hours, time zone (Abids today)
Coordinators	Give roles, switch a login off, duty hours, mentees, treasurer and Ishtagoshti-editor flags
All students and *Import students*	The whole list; import up to 500 adults from Excel or CSV, each row on its own; undo an import
Form choices, *How students found us*	The option lists of the sign-up and About you; where students heard of the class
Change history	The audit log: who changed a student, call, tick, lesson, setting… and when
Data requests, *Sign-in codes*, *App errors*	Privacy requests (section 15); staff sign-in codes **NEXT**; what broke in the app, without names **NEXT**

The staff home (`screens/guru-home.tsx`, `coordinator-home.tsx`) shows today's numbers and what waits for the person in one request (`staff_dashboard()`, 0050), and a ring of module circles around the drum (`components/module-ring.tsx`).

13. Parents by email, and Ishtagoshti for everyone

Check-out email to a parent OFF (0043, 0044)

When a student under 18 is checked out in the app, the parent can get a short email. Nothing is sent until the Guru switches it on in *Settings* (`parent_notices_enabled` is false by default; the check-in email and the "no check-out recorded" night email are separate switches, also off).

1. A trigger on `visits` (`visits_parent_notices`) puts a row in the queue `parent_notices` for every path: QR, tap, Check out all.
2. The job `mridanga-parent-notices` (every minute) calls `notify-parents`, which claims rows, sends through Brevo, and records each result; retries after 2, 8, 18, 32 minutes; at most 200 a day.
3. The email has a one-click **unsubscribe** link, signed so nobody can forge it; a stop on one address stops it for every child.

The queue holds no location, distance, photo or text, is closed to every app role, and is deleted after 30 days.

Weekly email to parents OFF (0058)

Opt-in per parent: the desk records the parent's written yes as a consent of scope `digest`. When the Guru switches it on, the job `mridanga-parent-digests` queues the week on Sunday evening and sends between 07:00 and 21:30: visits, practice minutes against the goal, ticked items, a level change, the next event. Never a location, a photo or another child. Staff can preview it on the student's page (`preview_parent_digest`).

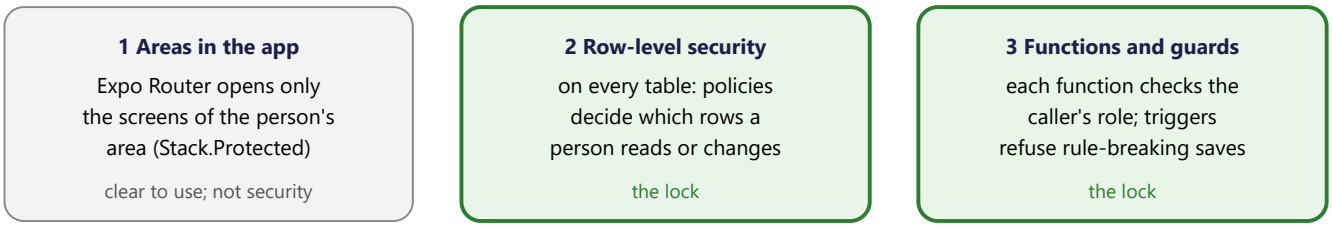
Ishtagoshti: sloka study (0021, 0027)

- The **Slokas** tab: themes, the *sloka of the day* (the same for everyone on an India date), each sloka in Devanagari, transliteration and a translation and purport in English, Telugu or Hindi; recitations; private notes; memorised ticks.
- **Copyright**: only the temple's own translations; the editor ticks "the temple's own, not BBT" and the database refuses to publish without it. Editors are coordinators the Guru marks (`profiles.ig_editor`).
- **Free public sign-up**: anyone can *Join Ishtagoshti* and read the slokas, and nothing of the class. It is not a new role: a pending login with a row in `ig_subscribers`; the function `ig_reader()` opens the slokas to it. Under 18, it reads only after the parent typed the code emailed to them. The test net runs as such a member over every table, bucket and function.

> Both email features and the Ishtagoshti parent code go out through Brevo. The Brevo key lives in the Supabase Vault and the Edge Function's secrets, never in the app or the repository.

14. Security: roles, row-level security, keys

Three layers, only one of them is the lock



Roles and areas

`src/auth/auth-provider.tsx` works out the person's **area** from the login and its `profiles` row: `signedOut`, `recovery` (set a new password), `signInCode`, `pending`, `guru`, `coordinator`, `student`, `subscriber`, or `loading`. `src/app/_layout.tsx` opens only that area's screens; the `staff/` folder is shared by the Guru and coordinators. When the database refuses a call as not allowed, the app reads the profile again, so a login switched off while the app is open moves to "Account switched off" at once.

Rules every maintainer must know

- **RLS is on for every table**; the test net fails if a new table has it off. Policies call their role helpers as `(select is_staff())` so Postgres asks once per query, not once per row.
- **Views read as the person asking** (`security_invoker = true`), so RLS still applies under them.
- **Every function is revoked from everyone and granted only to the roles that need it**; role checks are NULL-safe, so a switched-off login is refused like a stranger. Someone not signed in (`anon`) may run exactly one function, `sign_up_choices()`.
- **Nobody gives themselves a role.** Only the Guru gives roles; the Guru role itself is set only in the Supabase dashboard.
- **A child's record has no side doors**: the login link, QR token and consent fields are frozen for app users; consents are insert-only and checked at commit.

Keys: what is public and what is secret

Public (in every copy of the app): the project's URL and its **publishable** key. Fine to ship: the key only opens the front door; RLS decides everything after it.

Secret (never in the app, the repo or a chat): the `service_role` key, which skips every rule. Only the two Edge Functions use it, inside Supabase. The app **refuses to start** if it finds it in `.env` (`src/lib/supabase.ts`), and the publish scripts scan the bundle for it.

Staff sign-in codes NEXT (0061, DECISIONS #580-#589)

A Guru or coordinator can add a 6-digit code from an authenticator app (Supabase Auth's TOTP). Once every staff member has one, the Guru may switch `staff_mfa_required` on: then parents' details, consents, call notes and the logs are read only by a session that gave the code. The switch refuses to lock out the only Guru. A lost phone: the Guru resets the code in *Sign-in codes*.

Also: the web version sends strict security headers (a Content-Security-Policy naming only its own Supabase address, HSTS, nosniff), written by the export script from `app/public/_headers`. Every sign-out clears the QR card, notifications, cached photos and picked files from the device (`src/lib/device-traces.ts`).

15. Privacy and the law (DPDP)

India's **Digital Personal Data Protection Act 2023** and its **Rules 2025** apply. In plain words: tell people what you keep and why (a privacy notice), get valid consent (for a child, a parent's **verifiable** consent), keep only what you need, let people see, correct and erase their data, delete it when it is no longer needed, protect it, and report a breach. The app is built around that.

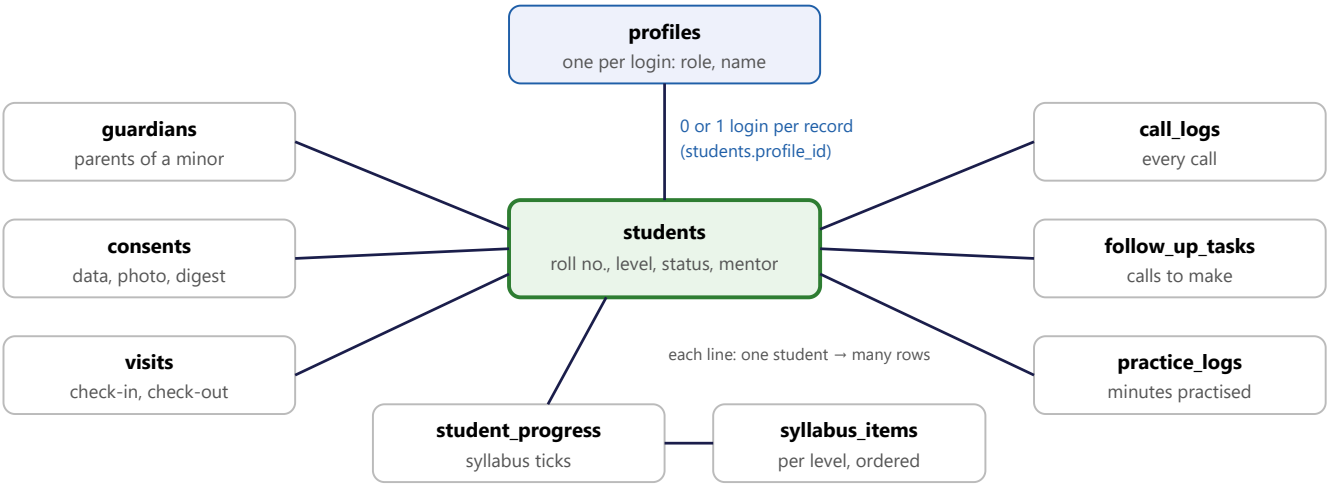
Principle	How the app does it	Where
Keep the minimum	Area and pincode, not the full address; only the **kind** of ID seen, never its number; the location **result**, never the position	DECISIONS #8, #70
Consent before data	A consent register (`consents`): scope `data`, `photo`, `digest`; method written or email code; who verified, when, which notice version. Insert-only, checked at commit	0003, 0025, 0051
The privacy notice	On the website (`/privacy/`), linked at sign-up with a required tick. Version 1.0 NEXT: the homes ask each adult to accept it once per version; a child is asked to show it to the parent (`notice_acceptances`)	0060, `lib/privacy-notice.ts`
Who read private details	Reads of parents' contacts, consents and call notes go through logging functions into the **access log**, which only the Guru reads; kept 400 days	0034
Who changed what	The **change history** (`audit_log`) records changes to students, logins, calls, ticks, lessons, settings…	*Change history*
See and ask	*My data and rights* (My profile): shows what the class holds, downloads it, and files a request: correction, erasure, withdrawal, complaint, nominee. Answered within **30 days**. A child's parent asks at the desk	0059, *Data requests*
Withdraw consent	`withdraw_consent` (Guru): the record is frozen, the login switched off, consents revoked, nothing new recorded	0025
Erase	`erase_student` / `erase_login`: the record, the login, files and audit copies go; a **tombstone** (roll number, date, reason, reference) stays in `erasures` as proof	0025, 0049
Do not keep for ever	NEXT Nightly job `mridanga-retention`: erase 30 days after a withdrawal, two years after leaving, a never-registered sign-up after 180 days; change log after two years	0060
Children apart	Under 18: parent consent (section 6), a child cannot file a request in the app, parent emails only for minors and only when switched on	0047, 0051
Data outside India	LIVE is in Sydney (Australia), TEST in Singapore; allowed while India restricts no country; named in the notice with every processor	OPERATIONS.md "processors"

> **For maintainers:** no real personal data in code, tests, seeds or screenshots (many students are minors: use the fictional seed students); never add a column that stores an ID number, a precise location or a face; never log names with errors (the error log keeps only the error name, the screen pattern, the version and the platform). A new kind of data about a child means a line in the privacy notice first. Backups hold children's data: always encrypted, never in the repo, a chat, WhatsApp or email.

Runbooks for each request, withdrawal, erasure and a data breach: `docs/OPERATIONS.md` ("Data requests", "Consent withdrawal", "Erasure request", "Retention", "Incidents → Personal data breach"). Privacy questions: privacy@mridangaseva.com.

16. The data model: tables, migrations, functions, jobs

The core tables and how they connect



A student record can exist **without** a login (many students never install the app). Everything about a student points at `students.id`; the record's link to a login is `students.profile_id`. The view `student_overview` adds the last visit, days since, and "here now" for the lists.

Tables by area (over 70 in all)

Area	Tables
People and settings	`profiles`, `person_details`, `choice_options`, `settings`, `centres`
Students	`students`, `guardians`, `consents`, `roll_counters`, `minor_signups`
Attendance and follow-up	`visits`, `call_logs`, `follow_up_tasks`, `status_history`
Learning	`levels`, `syllabus_items`, `student_progress`, `level_history`, `materials`, `assessments` (+ releases, assignments, submissions), `promotion_nominations`, `promotion_feedback`
Practice	`taals`, `practice_logs`, `practice_plans`, `practice_goals`, `practice_mantras`, `practice_links`
Communication	`announcements` (+ reads, replies), `groups`, `group_members`, `notifications`, `push_tokens`, `push_queue`, `push_outbox`, `events` (+ rsvps, performers, attendance), `polls`, `poll_votes`
Running the class	`fund_entries`, `fund_categories`, `inventory_items` (+ loans, checks, stocktakes), `duty_shifts`, `duty_assignments`
Ishtagoshti	`ig_slokas`, `ig_themes`, `ig_theme_slokas`, `ig_daily_pins`, `ig_notes`, `ig_memorised`, `ig_subscribers`, `ig_parent_codes`
Parents	`parent_notices`, `parent_digests`, `parent_mail_log`
Privacy and logs	`audit_log`, `access_log`, `erasures`, `data_requests`, `notice_acceptances`, `client_errors`

Every table and non-obvious column carries a `COMMENT ON` description, readable in the Supabase dashboard (Table Editor → table). The full description of every table, function and rule is `docs/DATABASE.md`.

The migrations, 0001 to 0061

59 files (numbers 0012 and 0041 were never used). Grouped by what they brought:

Files	What they brought
0001-0011	Phase 1 (Sept-Oct 2026): tables, RLS, login linking, registration with consent, attendance, follow-up, syllabus ticks, announcements, home screens, files, push
0013-0015	Syllabus editor and lessons, the Guru's admin screens, inbox, reports, centres
0016-0023, 0026, 0027	Phase 2, built early: assessments, promotion, practice, media, Ishtagoshti, events and polls, team tools, fund, public Ishtagoshti
0024	Location check at check-in
0025, 0028, 0030, 0037, 0039	Security and audit rounds: NULL-safe role checks, frozen fields, the consent register, withdrawal and erasure, anon locked out, per-query role helpers
0029, 0031-0033, 0040, 0042	Comments, push queue per phone, sign-in fixes, time zones and currency, operations clean-ups
0034, 0035, 0036	Access log; instrument QR labels and stocktake; account creation and About you
0038, 0043-0049	Correctness fixes; parent check-out emails; printed QR cards; Accept at the desk; form rules in SQL; withdrawn records; launch clean-up
0050-0054	Around the launch (20-10-2026): staff dashboard, a child's sign-up at home, practice of the week, coordinators' own records, after-launch forms
0055-0059	Learn more links, practice extras, practice while waiting, weekly parent email, data rights. **LIVE has run up to 0059** (24-10-2026)
0060, 0061 NEXT	Privacy notice 1.0 and retention; staff sign-in codes and the error log. On main and TEST, waiting for the next LIVE release

Phase 2 files were renumbered when they merged into main, so the TEST project ran a few under their branch numbers; LIVE runs every file in number order. `docs/GO_LIVE_CHECKLIST.md` part B lists each LIVE run with its probe.

Functions the app calls (the most used)

Function	Who	Does
`scan_qr`, `mark_visit`, `check_out_all`	staff	Attendance (section 7)
`register_student`, `accept_sign_up`, `accept_minor_sign_up`	staff	Registration (sections 5, 6)
`log_call`	staff	Records a call and applies its outcome
`get_guardians`, `get_consents`, `get_call_notes`, `get_audit_log`	staff / Guru	Private reads, each logged in the access log
`student_home`, `staff_dashboard`, `class_report`	by role	Home-screen numbers, reports
`log_practice`, `practice_days`, `set_my_practice_goal`	student	Practice log and weekly goal
`my_data_summary`, `file_data_request`	the person	My data and rights
`withdraw_consent`, `erase_student`, `erase_login`	Guru	Withdrawal and erasure

Errors are short `snake_case` codes (`not_allowed`, `minor_needs_consent` ...) that the app turns into messages in the person's language; the smoke test fails if a function of ours raises a sentence with a space in it.

Scheduled jobs (pg_cron; India time)

Job	When	Does
`mridanga-push`, `mridanga-parent-notices`	every minute	Call the Edge Functions when a push or a parent email waits
`mridanga-parent-digests`	every 5 minutes	The weekly parent email, while switched on
`mridanga-close-visits`	every hour	Close visits left open after closing time
`mridanga-status-refresh`	06:00	Irregular / Inactive, call tasks, ended pauses
`mridanga-assessments`, `mridanga-events-polls`	09:00	Reminders; deletes old recordings
`mridanga-duty`	18:00	Reminds tomorrow's roster
clean-ups: inbox, access log, orphan files, cron history, app errors	06:30-07:45	Delete what has passed its keep time
`mridanga-minor-signups`, `mridanga-data-requests`, `mridanga-retention` NEXT	02:15-02:45	Expire unanswered child sign-ups; remind data requests; retention

pg_cron's own times are in UTC (06:00 India time = 00:30 UTC); `docs/DATABASE.md` "Scheduled jobs" lists both, and the SQL in `docs/OPERATIONS.md` "Scheduled jobs: health check" shows each job's last runs.

Storage buckets (all private)

Bucket	Holds	Largest file	Rule in
`announcement-files`	Photos and PDFs on News posts; opens only for those who may read the post	5 MB	0010
`material-files`	Lesson PDFs and photos	10 MB	0013
`assessment-files`	Assessment files and students' recordings; deleted 30 days after review	50 MB	0016, 0049
`ishtagoshti-audio`	Sloka recitations	10 MB	0021
`fund-bills`	Bills on fund entries; a bill on an entry cannot be deleted	10 MB	0026

A file is never opened by a public address. The app asks for a **signed link** that works for a short time, and Storage checks the same kind of row-level rules as the tables (for example `announcement_file_readable`) before it gives one.

Views

A **view** is a saved query that looks like a table. Ours (`student_overview`, `announcement_audience`, `announcement_seen`, `announcement_reply_list`, `group_summary`, `assessment_tracker`, `promotion_queue`...) are made `with (security_invoker = true)`, so they read the tables as the person asking and row-level security still applies. Without it, a view runs as its owner and would show every student to anyone signed in (DECISIONS #20).

17. Code map: which folder does what

```
mridanga-seva/
  app/                        the Expo app (Android + web)
    app.json                  name, icons, permissions, plugins (part of the fingerprint: changing it = new APK)
    eas.json                  build profiles preview (TEST) and production (LIVE) = update channels
    .env.example              the two settings: Supabase URL + publishable key (.env files are never in git)
    public/_headers           web security headers (template)
    scripts/                  export-web.mjs, publish-update.mjs, eas-build-guard.mjs, bundle-checks.mjs,
                              typed-routes.mjs, staff-screens.mjs, service-worker.js, lib/supabase-projects.mjs
    tests/                    45 node:test files + stubs/ (pure helpers, release scripts, translations)
    src/
      app/                    SCREENS. Every file is a route (Expo Router); _layout.tsx arranges them
        _layout.tsx           opens only the signed-in person's area (Stack.Protected)
        other .tsx files      screens before a role: sign-in, sign-up, forgot- and reset-password,
                              sign-in-code, pending, about-you, parent-consent, join-ishtagoshti
        student/ (tabs)/      Home, My QR, News, Slokas; + progress, visits, practice, assessments/ …
        staff/   (tabs)/      Home, Check-in, Students, Follow-ups, News; + register, call/, fund/, …
        subscriber/           the public Ishtagoshti member's screens
        waiting/              While you wait: practice, mantra, Beginner lessons
        i/                    /i/<token>: an instrument label opened by any camera
      screens/                big screens shared by areas (staff homes, practice tools, lesson player,
                              Ishtagoshti, events, polls, My profile, My data and rights …)
      components/             building blocks: buttons, fields, cards, QR scanner, module ring, two-head view …
      data/                   READING AND SAVING RECORDS, one file per area (students.ts, attendance.ts,
                              fund.ts …) with the form checks. Screens call these, never Supabase directly
      auth/                   who is signed in, their role and area (auth-provider.tsx), sign-in calls, mfa.ts
      lib/                    helpers: supabase.ts (the client), dates, money, phone numbers, QR reader,
                              push, app updates, offline queue, practice audio and games, CSV, error report
      i18n/                   index.ts + locales/en.json, te.json, hi.json (every word the app shows)
      theme/                  colours, spacing, text sizes, light and dark
      content/                whats-new.json (the What's new notes)
      web-routes/             the web version's screen list; staff screens load as one extra file
supabase/
  migrations/                 0001_phase1.sql … 0061_staff_mfa.sql, run in number order
  functions/                  Edge Functions: notify-announcements/ (push), notify-parents/ (emails)
  tests/                      smoke-test.mjs (2,224 checks), push-messages, parent-notices, live-replay
  seed.sql                    15 fictional students for TEST. Never on LIVE
website/                      the public website mridangaseva.com (content/en|te|hi, src/build.mjs)
design/                       design tokens (colours) shared by app and website
docs/                         see below
.github/                      workflows/ci.yml (CI), dependabot.yml
```

The documents (`docs/`)

File	Read it for
`ARCHITECTURE.md`	How the pieces fit; each flow step by step
`DATABASE.md`	Every migration, table, function, job and "who can see what"
`DECISIONS.md`	595 numbered decisions with their reasons (section 21)
`OPERATIONS.md`	Runbooks: builds, updates, web upload, push, emails, backups, incidents, handing over
`GO_LIVE_CHECKLIST.md`	The ordered steps that took LIVE to the launch, and each later LIVE migration
`SCREENS.md`	Every screen with its code and status
`TRANSLATIONS.md`, `I18N.md`, `GLOSSARY.md`	Languages; dates, money, time zones; the words
`WHY_THIS_STACK.md`, `guide/`	For people who ask; the user guide for students, coordinators, the Guru, new helpers
`phase3/`	Face attendance plans, consent and DPIA drafts (Phase 3)

Start reading code at a screen in `src/app/`, follow its imports into `data/` (what it asks the database), then look the function up in `DATABASE.md`. Every source file starts with a comment saying what it is for.

18. Screens and their files

The screen's name as the app shows it (English), where to find it, its code in the docs, and its file under `app/src/app/`. Shared screens live in `app/src/screens/` and the route file only passes the area.

Screen (where)	Code	File
Sign in · New student registration · Reset your password	A1	`sign-in.tsx`, `sign-up.tsx`, `forgot-password.tsx`, `reset-password.tsx`
Waiting for access · While you wait · About you	A2w, A4	`pending.tsx`, `waiting/`, `about-you.tsx`
Consent for your child (parent, no login)	–	`parent-consent.tsx`
Home (student)	S1	`student/(tabs)/index.tsx`
My QR card (My QR tab)	S3	`student/(tabs)/my-qr.tsx`
My progress · My attendance	S4, S9	`student/progress.tsx`, `student/visits.tsx`
Practice tools · My practice log	S5, S6	`student/practice.tsx`, `practice-log.tsx`
My profile · My data and rights · Notifications	A3, A3d, A2	`student/profile.tsx`, `my-data.tsx`, `notifications.tsx` (same in `staff/`)
Home (Facilitator / coordinator)	G1, C1	`staff/(tabs)/index.tsx` → `screens/guru-home.tsx`, `coordinator-home.tsx`
Welcome desk (Students tab)	C2, C3	`staff/register.tsx`
Check-in (tab) · In class now	C5, C6	`staff/(tabs)/attendance.tsx`, `mark-attendance.tsx`, `here-now.tsx`
Students (tab) · a student's page	C7, C8	`staff/(tabs)/students.tsx`, `staff/students/`
Mark progress (from a student's page)	C9	`staff/syllabus/`
Follow-ups (tab) · Record a call	C10, C11	`staff/(tabs)/follow-up.tsx`, `staff/call/`
News (tab) · Groups	C15, S10	`staff/(tabs)/announcements.tsx`, `staff/announcements/`, `staff/groups/`
Reports / My reports	G8, C21	`staff/reports.tsx`
Assessments · Level-ups	G6, C12-C14, S7, C22, C23, G7	`staff/assessments/`, `student/assessments/`, `staff/promotion/`
Lessons (syllabus editor) · Rhythms (taals)	G4, G5	`staff/levels/`, `staff/materials/`, `staff/taals/`
Events · Polls	C16, C17, S11, S12	`staff/events/`, `staff/polls/` (and in `student/`)
Class fund · Instruments · Seva roster · Suggestions	F1-F7, C19, C20, C18	`staff/fund/`, `staff/inventory/`, `staff/duty/`, `suggestions.tsx`
Print QR cards	C24	`staff/qr-cards.tsx`
Slokas (tab) · Ishtagoshti members	I1-I3, I11, I12, I15	`(tabs)/ishtagoshti.tsx`, `staff/ishtagoshti/`, `screens/ishtagoshti-*.tsx`
Set-up: Settings · Centres · Coordinators · All students · Change history	G10, G9, G2, G3, G11	`staff/settings.tsx`, `centres/`, `coordinators/`, `database/`, `audit-log.tsx`
Data requests · Sign-in codes · App errors	G14, G15, G16	`staff/data-requests/`, `sign-in-codes.tsx`, `app-errors.tsx`
Weekly email to parents · What's new	–	`staff/parent-digest.tsx`, `whats-new.tsx`

> **Tabs.** Student: Home, My QR, News, Slokas. Staff: Home, Check-in, Students, Follow-ups, News (bottom bar on a phone; a sidebar from 900 px wide, which also holds Slokas). Other screens open on top of the tabs with a back button; every Back calls `goBackOr(parent)`, so a screen opened from a notification never leaves a blank page.

19. Testing and CI

Run these before every commit

Where	Command	What it checks
`app/`	`npx expo lint`	Common coding errors and unsafe patterns
`app/`	`node scripts/typed-routes.mjs` then `npx tsc --noEmit`	Types; every `router.push` path against the real screens; a word missing in Telugu or Hindi
`app/`	`npm test`	45 test files under Node's test runner: dates, phone numbers, names, translations (same keys and placeholders in en / te / hi), offline queue, release scripts, repo guards ...
`supabase/tests/`	`npm ci` then `npm test`	The smoke test, the push and parent-email tests, and the LIVE replay (below)
`website/`	`npm test`	The website builds; links, languages, headers

The smoke test: a whole database on your laptop

`supabase/tests/smoke-test.mjs` starts **PGlite** (PostgreSQL compiled to WebAssembly, in memory: no server, no Supabase account), imitates Supabase's `auth` schema, roles and Storage, runs **every migration** and `seed.sql`, then tries what each kind of person might do and checks the database allows or refuses it, with the **exact** error code. It ends with `All 2224/2224 checks passed`, and fails if the count differs from `EXPECTED_CHECKS`, so a skipped block cannot pass unseen.

After the per-migration sections comes the **test net**: it walks the whole schema: every table has RLS; every function the app may run is on an allow-list (`SIGNED_IN_MAY_RUN`); a student-isolation loop over every column that names a student; write sweeps per role; triggers and jobs.

> **The LIVE replay** (`live-replay.mjs`): builds a database at LIVE's old state with rows of every kind, applies the later migrations in order, runs each file **twice** (the second run must change nothing), and compares the result with a fresh build. A new migration fails it until the go-live checklist lists it.

> **What PGlite cannot prove**: real pg_cron timing, pg_net calls, Storage size limits, two sessions at once, Supabase's own MFA. These are listed at the top of the smoke test and checked by hand on TEST.

CI: GitHub Actions (`.github/workflows/ci.yml`)

On every pull request and every push to `main`, three jobs run with Node 24: **App** (npm ci, lint, typed routes, tsc, npm test), **Database** (the whole `supabase/tests` npm test) and **Website**. They need no secrets and only read the code. Actions are pinned by commit SHA; Dependabot proposes updates weekly (npm) and monthly (actions). A red run: open the failed step, read the first `FAIL` line, fix on the branch, push again.

> **CI is not everything.** It does not run the app on a phone, deploy, publish or touch TEST/LIVE. Screens have no automatic tests yet: check a changed screen in a browser as each role (phone width 375 px and laptop 1280 px) on TEST, and describe how in the pull request. "It builds" is not "it runs".

Adding a test

- A pure helper in `app/src/` → a `node --test` file in `app/tests/` (a native package gets a stub in `tests/stubs/`).
- A migration → a new section in the smoke test: the roles that must be refused (`refusesWith(name, 'exact error', fn)`) and the role it is for; then set `EXPECTED_CHECKS` to the new total.
- A new function the app calls → add it to `SIGNED_IN_MAY_RUN`. A new table → RLS on; its `student_id` / `profile_id` columns are checked by the isolation loop on their own.

20. Release and operations

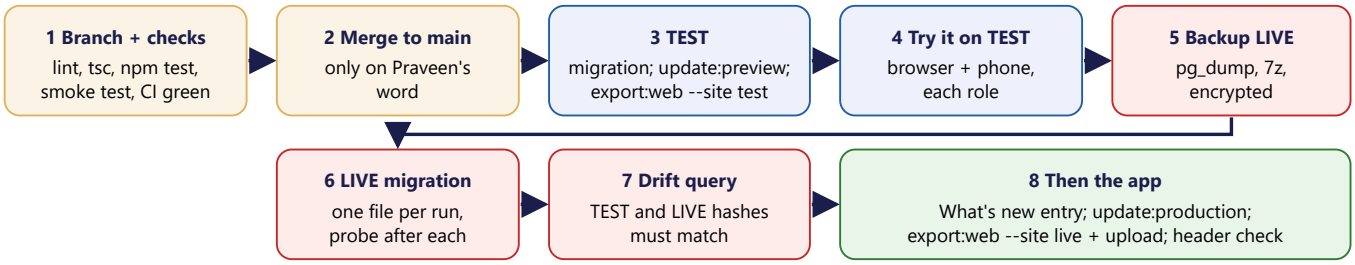
An APK has two layers

> **Build (a new APK)** = the whole app, native code included. Made on Expo's servers (EAS Build), free queue up to an hour or more, 15 a month. Needed when anything native changes: a package with native code, an Expo SDK upgrade, `app.json`, `eas.json`, the icon. Everyone installs it once.

> **Update (OTA, "over the air")** = only our JavaScript: screens, words, colours, logic. Published in minutes; the app downloads it at start and offers **Restart now**. Reaches only APKs with the same **fingerprint**.

The **fingerprint** is a hash Expo computes from everything native. The class's production APK (build `90c6d1cd`, 16-10-2026) has fingerprint `23dc7e4a`. Check yours with `npx expo-updates runtimeversion:resolve --platform android` in `app/`: unchanged → an update is enough. That is why native changes are grouped for one planned APK, and why face scan, with its native libraries, waits on its own branch.

Database first, then the app (DECISIONS #121)



Why this order: an app update reaches phones within minutes, so if it calls something LIVE's database does not have yet, the class's app breaks. And **migrations stay additive for one release**: never drop or rename what the app in use calls, so a bad update can always be rolled back (`npx eas-cli update:rollback`) without touching the database.

The guards in the scripts

Script	What it refuses
`npm run update:preview` / `update:production` (`scripts/publish-update.mjs`)	A network that breaks uploads; for production, anything but a clean `main` equal to `origin/main` with a dated What's new entry; no APK with this fingerprint; settings of the wrong project; a bundle holding a secret. Then you must type `yes`. `--check-only` runs every check and publishes nothing
`npm run export:web -- --site test|live` (`scripts/export-web.mjs`)	No `--site` (it never guesses); a URL that is not that site's project; a secret in the export. Writes `_headers`, `version.txt`, the offline service worker
`scripts/eas-build-guard.mjs` (runs on EAS before every build)	A `preview` build not pointed at TEST, or a `production` build not pointed at LIVE

Running it

- **Backups**: every Monday and before every LIVE migration: three `pg_dump` files (schema, data, logins), encrypted with 7-Zip, last 8 kept, a line in `backup-log.csv`, a written restore drill. The free plan has no downloadable backups.
- **APKs**: kept in the team's Drive folder with their SHA-256 (EAS links expire after about 14 days); the package name `org.mridangaseva.app` and the signing keystore must never change.
- **Office network**: its firewall breaks uploads to Expo; builds and updates go from a phone hotspot.
- **Monthly**: usage against the free limits; scheduled-jobs health check; Dependabot pull requests; `npm audit` triage.

Every step with its exact commands: `docs/OPERATIONS.md` ("Building the Android app", "Updating the Android app", "Publishing the web version", "Releasing a change: database first", "Backups") and `docs/GO_LIVE_CHECKLIST.md`.

21. How decisions are recorded

Every important choice is a numbered entry in `docs/DECISIONS.md`: **Context** (the situation), **Decision**, **Why**, **Consequences** (what it means for the code and for maintainers). Entry #1 is "React Native (Expo) and Supabase" of 28-09-2026; on 24-10-2026 the newest is #595. Newest at the bottom.

The rules of the log

- **Never rewrite** an entry. If a decision changes, mark the old one *Replaced by #N* and add a new one.
- Part changed: a line *Replaced in part by #N* under the old entry. A date or typo: an *Erratum* line.
- Code points at its reason: `// Paused/Left only via a call log, docs/DECISIONS.md #4`.

Entries worth reading first

- #1 Expo and Supabase · #2 attendance is a visit, not a roll call
- #3 roll numbers frozen · #4 Paused/Left only by a call
- #5 rules live in the database · #13 link only after email confirmation
- #16 a minor only with consent · #17 the QR holds a secret token
- #121 database first · #399 a child's sign-up at home

How the app is built

The app is built as seva by a volunteer maintainer (Praveen) working with **Claude**, an AI coding assistant. The maintainer and the class team decide *what* is built and approve every step; separate working sessions write code and documents on their own branches. Nothing reaches the class without the checks of section 19 and the maintainer's word: **"push to main"**, **"publish"**. You will see this in the git history: small branches, merge commits, and a decision entry and documentation with each change.

A change is not finished until its documentation is (`docs/README.md`). Behaviour changed → `DATABASE.md` or `ARCHITECTURE.md`; a new rule or trade-off → a DECISIONS entry; a new screen → `SCREENS.md`; a new term → `GLOSSARY.md`; something users see → a line in `app/src/content/whats-new.json` before the next production update.

Where the screen list came from

Before any code, on 28-09-2026, every screen the app might need was listed for the team (about 80, each with a code like A1 = login, C5 = mark attendance, G4 = syllabus editor) with **Approve / Change / Drop**. The team's answers shaped the app: three roles, no batches, three levels, email login, English first with Telugu and Hindi, parental consent, and Phase 1 must cost nothing. The codes live on in the docs and code comments.

22. How to contribute safely

The three rules that protect the children's data

1. **Never LIVE.** Never run anything against the LIVE project (`qeozvv…`). The main checkout's `app/.env` on the maintainer's PC points at LIVE: work in your own clone or a worktree whose settings point at TEST (`fhuqyk…`). A deploy command is never a test: on a PC where the Supabase CLI is logged in, it really deploys.
2. **Never real data.** Only the fictional seed students in code, tests, seeds and screenshots.
3. **Never secrets.** No `.env` file, `service_role` key, password or keystore in git or a chat. `.gitignore` refuses the usual ones.

Your first change, step by step

1. **Agree first.** Open or pick an issue on GitHub (Praviadari/mridanga-seva); for anything larger than a small fix, agree the approach before coding.
2. **Set up.** Node 24 (`.nvmrc`). Clone, `cd app` , `npm ci` ; copy `.env.example` to `.env.development` with the TEST URL and publishable key; `npx expo start` and press `w` for the web version. In `supabase/tests` : `npm ci` , `npm test` .
3. **Branch** from `main` (e.g. `fix/attendance-checkout`). Never commit to `main` directly.
4. **Change, following the house rules**: words only in `en.json` + `te.json` + `hi.json` (never in a screen); screens call `data/` , never Supabase directly; a database change is a **new** migration with RLS, comments, revoked-then-granted functions and smoke checks; Expo packages with `npx expo install` ; a header comment on every file; "prefer boring code".
5. **Check**: the commands of section 19, all green; the screen tried on TEST as each role it concerns, at phone and laptop width.
6. **Native or not?** If you added a package or touched `app.json` / `eas.json` , compare the fingerprint. A change of fingerprint means a new APK: say so in the pull request, it is merged only when a build is planned.
7. **Document**: DECISIONS entry, DATABASE / ARCHITECTURE / SCREENS lines, What's new if users see it.
8. **Pull request**: one change, a clear description, screenshots for a screen, how you checked it. CI must be green. Merging, pushing to main, running a LIVE migration and publishing stay with Praveen.

Common traps (from real mistakes)

Trap	What to do
A function raises a sentence instead of a code	Raise a short `snake_case` code; explain in `detail` ; translate it in the app
New function callable by everyone	Supabase grants new functions to PUBLIC: revoke from `public, anon, authenticated` , grant only what is needed
Editing a migration that already ran	Never. Add the next number. Two branches may both use a placeholder number; the lead renumbers at merge
Calling a new database function from an update before LIVE has it	Database first (section 20); or make the app fall back when the function is missing
A route path typo passes tsc on a fresh clone	Run `node scripts/typed-routes.mjs` before `npx tsc --noEmit`
Uploads to Expo fail half-way	The office network: use a phone hotspot. Never switch certificate checks off
"It works in development"	Test the real APK or the exported web version too; never hide an error from the user

23. What is not built yet, and Phase 3

State on 24-10-2026

Item	State
Privacy notice 1.0 card, retention job (0060); staff sign-in codes and the error log (0061); offline door and practice queue; a lighter web version	NEXT on main and TEST; in the next LIVE release (What's new entry waiting, date not set)
Check-out emails and the weekly email to parents	OFF built and on LIVE; waiting for the Guru's switch
The door tablet (role `kiosk`)	Not built; a coordinator's phone scans at the door
Push notifications on the web version (iPhones)	Not on main; being worked on separately
Play Store and App Store	Not chosen yet; Android installs from a link
The domain `app.mridangaseva.com` for the web app	Waits for the domain move (go-live checklist part E)
Telugu and Hindi texts	Careful drafts, to be checked by native speakers (review sheet in `docs/translation-review/`)
Real khol sounds for the practice tools	Prepared, waiting for the Guru's recordings (`docs/KHOL_RECORDINGS.md`)
Service accounts (Expo, Firebase, Cloudflare, Supabase)	On the maintainer's personal account until a team account exists (OPERATIONS "Handing over")

Phase 3: face attendance, on its own branch

Face attendance (a student checks in by looking at their own phone, only with a separate written consent) is **not on main**. It lives on the branch `face-scan` and runs only in a separate "Mridanga Seva TEST" app against the TEST project, because its native face libraries change the app's fingerprint: merging it early would stop updates reaching the class's phones. Its own explainer, made the same way as this one, is `docs/phase3/print/face-scan-explained.pdf` on that branch. The plans and legal drafts are in `docs/phase3/` on main.

> **Where to start as a new maintainer.** Read `docs/guide/08-new-helper.md`, then `ARCHITECTURE.md`, `DECISIONS.md` (#1-#30 first) and `DATABASE.md`. Run the app on TEST as a student and as a coordinator. Then pick a small issue, and follow section 22.

24. Glossary

APK	The Android app file people install from a link.
Area	The set of screens a person may open: signed out, pending, student, staff (Guru and coordinator), subscriber…
Audit log / Change history	The database's record of who changed what, and when.
Access log	The record of who *read* a child's private details (parents, consents, call notes).
Bol	A spoken syllable for a drum stroke (dha, ta, ghe…); a taal is a cycle of bols.
Bucket	A folder in Supabase Storage with its own rules (ours are private).
CI	Continuous integration: GitHub runs the checks on every pull request and push to main.
Consent	A row in `consents`: what was agreed (data, photo, weekly email), by whom, how, when, which notice version.
DPDP	India's Digital Personal Data Protection Act 2023 and its Rules 2025.
Edge Function	A small server program on Supabase: `notify-announcements`, `notify-parents`.
EAS Build / Update	Expo's cloud services that build the APK and send JavaScript updates.
Erasure / tombstone	Deleting a person's data for good; the tombstone keeps only the roll number, date, reason and reference as proof.
Expo / React Native	Tools to write one TypeScript app for Android, iPhone and the web.
Expo Router	Every file in `app/src/app/` is a screen and a web address.
Fingerprint	A hash of the app's native parts; an update reaches only APKs with the same one.
Facilitator	The Guru, as the English screens call the role `guru`.
Guard / trigger	A check the database runs when a row is saved; it can refuse the save.
i18n	"Internationalisation": every word in `en.json`, `te.json`, `hi.json`, fetched with `t('key')`.
Ishtagoshti	Sloka (verse) study; also open free to the public.
LIVE / TEST	The class's real Supabase project / the one with made-up data for building and testing.
Mentor	The coordinator who follows a student up with calls.
Migration	A numbered SQL file that changes the database; never edited once run.
OTA update	New JavaScript sent "over the air" to installed apps, without a new APK.
Pending	A login waiting for the desk; it reads only its own profile.
pg_cron	The timetable inside PostgreSQL that runs our 17 jobs.
PGlite	PostgreSQL that runs in memory inside Node; our tests use it.
Publishable / service_role key	The public key every app holds / the secret key that skips every rule (Edge Functions only).
Roll number	`MS-2026-0042`: given by the database, frozen, never reused.
RLS (row-level security)	Per-table rules in PostgreSQL deciding which rows each person may read or change.
Security definer / invoker	A function that runs with its owner's rights (and checks the caller itself) / with the caller's rights.
Seed	`supabase/seed.sql`: the fictional students of the TEST project.
Smoke test / test net	The 2,224 database checks run on PGlite; the net is the part that sweeps the whole schema.
Supabase	The hosted service around our PostgreSQL database: login, storage, functions, jobs.
Taal	A rhythmic cycle of beats; the practice player loops it.
Visit	One check-in and its check-out: a row in `visits`.
Worktree	A second working folder of the same git repository, on its own branch.

The full glossary, with the mridanga terms: `docs/GLOSSARY.md`. Code: github.com/Praviadari/mridanga-seva (MIT licence). This document: `docs/print/app-explained.html` (PDF made with Edge), facts as of main `b2e5fb5`, 24-10-2026.